

ASSOCIATION RULE MINING

Association rule mining is a technique in data mining for discovering interesting relationships, frequent patterns, associations, or correlations, between variables in large datasets.

It's widely used in various fields such as market basket analysis, fraud detection, healthcare analytics and recommendation systems.

The idea is to find rules that predict the occurrence of an item based on the occurrences of other items in the transaction.

There are many algorithms to implement Associate Rule Mining. Since Apriori algorithm is explained in previous reports, I will explain FP-Growth and ECLAT algorithms and provide example to show how they work.

First, lets give an simple example to show what is Association Rule Mining is.

Example:

Imagine a dataset where each transactions is listed with their items.

Transaction ID	Purchased Items
T1	Milk, Bread, Butter
T2	Bread, Diapers, Beer
T3	Milk, Diapers, Beer, Cola
T4	Bread, Milk, Diapers, Beer
T5	Bread, Milk, Diapers, Cola

Itemset is collection of one or more items in the dataset.

Single Itemset : {Bread}, {Milk}, {Butter}

Two-item Itemset : {Bread, Butter}, {Milk, Bread}

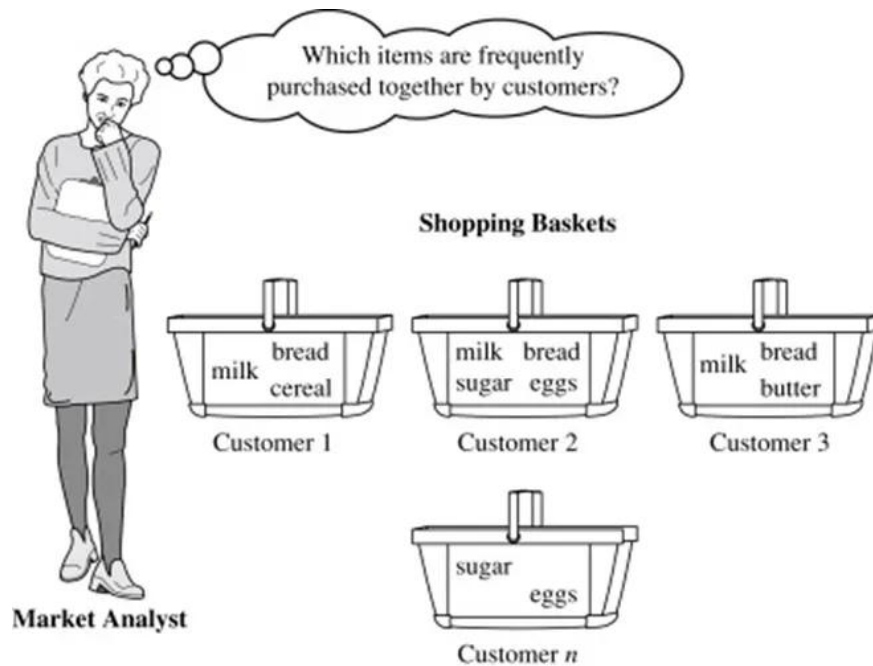
Three-item Itemset: {Bread, Diapers, Beer}

Association Rules

Association rule is a fundamental concept in data mining that reveals how items within a dataset are connected.

It explains relationship between two itemsets.

Imagine an association rule like {Bread} → {Cheese}. This means that user that bought bread is likely gonna buy cheese too.



Association rules have two sides: Antecedents and Consequents

$\{\text{Bread, Milk}\} \rightarrow \{\text{Beer}\}$

(Antecedent) (consequent)

Antecedent (X): This is the first part of the rule, the condition. It's the set of items found in the database that you are examining for patterns. In the rule $\{X\} \rightarrow \{Y\}$, X is the antecedent.

Consequent (Y): This is the second part of the rule, which is inferred from the presence of the antecedent in transactions. In the rule $\{X\} \rightarrow \{Y\}$, Y is the consequent.

There are few methods to evaluate association rules:

$$\begin{array}{l}
 \text{Rule } X \Rightarrow Y \begin{cases}
 \text{Support} = \frac{\text{Frequency}(X,Y)}{N} \\
 \text{Confidence} = \frac{\text{Frequency}(X,Y)}{\text{Frequency}(X)} \\
 \text{Lift} = \frac{\text{Support}}{\text{Support}(X) * \text{Support}(Y)}
 \end{cases}
 \end{array}$$

Support: It measures how frequently X and Y together appears in the dataset. It's the proportion of transactions that contain both X and Y to all transactions. We mostly put threshold value to be sure not take values with small support.

Confidence: It measures the chance of finding consequent in a transaction that also has antecedent.

Ex: percentage of users that bough item A also bought B.

Lift: It measures how much more often the antecedent and consequence of the rule occur together than we would expect if they were statistically independent.

Now, I will show association with real algorithms such as FP-Growth and ECLAT algorithms but first I need to explain these algorithms theoretically.

FP Growth Algorithm (Frequent Pattern)

The FP-Growth (Frequent Pattern Growth) algorithm efficiently mines frequent itemsets from large transactional datasets. Unlike the Apriori algorithm which suffers from high computational cost due to candidate generation and multiple database scans. FP-Growth avoids these inefficiencies by compressing the data into an FP-Tree (Frequent Pattern Tree) and extracts patterns directly from it.

It can be used in basket market analysis, fraud detection, and web usage mining.

How FP Growth Works

Imagine a small dataset:

Transaction ID	Items
T1	{E,K,M,N,O,Y}
T2	{D,E,K,N,O,Y}
T3	{A,E,K,M}
T4	{C,K,M,U,Y}
T5	{C,E,I,K,O,O}

First, we will compute how frequently each item appear in the dataset.

A:1 M:3
C:2 N:2
D:1 O:3
E:4 U:1
I:1 Y:3
K:5

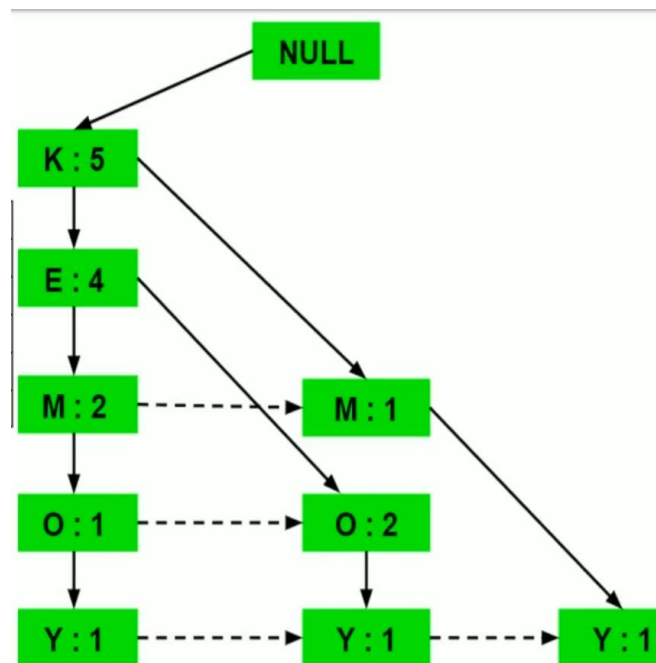
Lets give minimum support as 3. It means that each item must have at least appear 3 times or more.

After insertion item list will look like this: L: {K:5, E:4, M:3, O:3, Y:3}

Then, we will sort each transactions item list by their frequency.

Transaction ID	Sorted by Frequency
T1	{K, E, M, O, Y}
T2	{K,E, O, Y}
T3	{K, E, M}
T4	{K, M, Y}
T5	{K, E, O}

After looking through list, we will create an FP Tree.



Here is the list after every transaction is read.

Now, for each item, Conditional Pattern Base is computed which is path labels of all the paths which lead to any node of the given item in the frequent-pattern tree.

Items	Conditional Pattern Base
Y	{{K,E,M,O:1}, {K,E,O:1}, {K,M:1}}
O	{{K,E,M:1}, {K,E:2}}
M	{{K,E:2}, {K:1}}
E	{K:4}
K	

From the Conditional FP Tree, frequent pattern rules are generated by pairing items of the Conditional Frequent Pattern set to corresponding to the item as given in the below table.

Items	Conditional Pattern Base	Conditional Frequent Pattern Tree
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}	{K : 3}
O	{{K,E,M : 1}, {K,E : 2}}	{K,E : 3}
M	{{K,E : 2}, {K : 1}}	{K : 3}
E	{K : 4}	{K : 4}
K		

Items	Frequent Pattern Generated
Y	{<K,Y : 3>}
O	{<K,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{<K,M : 3>}
E	{<E,K : 3>}
K	

Here is a Python example with Online Retail Dataset:

```

online_retail_example.py × Online Retail.xlsx

1 import pandas as pd
2 from mlxtend.frequent_patterns import association_rules
3 from mlxtend.frequent_patterns import fpgrowth
4 from mlxtend.preprocessing import TransactionEncoder
5
6 pd.set_option('display.width', 1000)
7 pd.set_option('display.max_columns', 8)
8
9 df = pd.read_excel('Online Retail.xlsx')
10 print("First 5 rows of dataset")
11 print(df.head())
12 print("\n")
13
14 print("Dataset Information")
15 print(df.info())
16 print("\n")
17
18 print("Dataset Description")
19 print(df.describe())
20 print("\n")
21
22 print("Missing Values in the Dataset")
23 print(df.isnull().sum())
24 print("\n")

```

```

26 # Data Preparation
27 df = df.dropna(subset=['Description'])
28 df = df.dropna(subset=['CustomerID'])
29 df = df[df['Quantity'] > 0]
30 df = df[df['UnitPrice'] > 0]
31 df = df[~df['InvoiceNo'].astype(str).startswith('c')]
32 df['Description'] = df['Description'].str.strip()
33
34 transactions = df.groupby('InvoiceNo')['Description'].apply(list).tolist()
35 transaction_Encoder = TransactionEncoder()
36 transaction_Encoder.fit(transactions)
37 encoded_Transactions = transaction_Encoder.transform(transactions)
38
39 df_pd = pd.DataFrame(encoded_Transactions, columns=transaction_Encoder.columns_)
40
41 frequent_item_sets = fpgrowth(df_pd, min_support=0.01, use_colnames=True)
42 frequent_item_sets = frequent_item_sets.sort_values(by=['support'], ascending=False)
43
44 print("Frequent Items Sets")
45 print(frequent_item_sets.head())
46 print("\n")
47
48 rules = association_rules(frequent_item_sets, metric='lift', min_threshold=1.0)
49 print(rules[['antecedents', 'consequents', 'support', 'lift', 'confidence']].head(15))
50 print("\n")
51

```

Pandas library and mlxtend added to the project. Mlxtend is for adding fpgrowth() function and pandas is to add csv file to the project.

Dataset information is printed out and after that, Data Preparation applied so dataset is ready for FP-Growth.

TransactionEncoder() used to create True / False list so fpgrowth() can easily read.

Fpgrowth() is used an min_support defined as 0.01 and values sorted by their support descending.

Frequent item sets are printed and association rules are defined by lift.

Lastly, associations are printed with antecedents and consequents.

Output is in next page:

	antecedents	consequents	support	lift	confidence
0	frozenset({JUMBO BAG PINK POLKADOT})	frozenset({JUMBO BAG RED RETROSPOT})	0.029463	7.260672	0.626866
1	frozenset({JUMBO BAG RED RETROSPOT})	frozenset({JUMBO BAG PINK POLKADOT})	0.029463	7.260672	0.341250
2	frozenset({ROSES REGENCY TEACUP AND SAUCER})	frozenset({GREEN REGENCY TEACUP AND SAUCER})	0.029193	18.530185	0.690932
3	frozenset({GREEN REGENCY TEACUP AND SAUCER})	frozenset({ROSES REGENCY TEACUP AND SAUCER})	0.029193	18.530185	0.782923
4	frozenset({ALARM CLOCK BAKELIKE GREEN})	frozenset({ALARM CLOCK BAKELIKE RED})	0.028599	14.194548	0.671736
5	frozenset({ALARM CLOCK BAKELIKE RED})	frozenset({ALARM CLOCK BAKELIKE GREEN})	0.028599	14.194548	0.604333
6	frozenset({LUNCH BAG PINK POLKADOT})	frozenset({LUNCH BAG RED RETROSPOT})	0.028221	8.082737	0.561762
7	frozenset({LUNCH BAG RED RETROSPOT})	frozenset({LUNCH BAG PINK POLKADOT})	0.028221	8.082737	0.406056
8	frozenset({LUNCH BAG BLACK SKULL.})	frozenset({LUNCH BAG RED RETROSPOT})	0.027898	7.071006	0.491445
9	frozenset({LUNCH BAG RED RETROSPOT})	frozenset({LUNCH BAG BLACK SKULL.})	0.027898	7.071006	0.401398
10	frozenset({WOODEN PICTURE FRAME WHITE FINISH})	frozenset({WOODEN FRAME ANTIQUE WHITE})	0.025254	12.207962	0.534247
11	frozenset({WOODEN FRAME ANTIQUE WHITE})	frozenset({WOODEN PICTURE FRAME WHITE FINISH})	0.025254	12.207962	0.577065
12	frozenset({LUNCH BAG RED RETROSPOT})	frozenset({LUNCH BAG SPACEBOY DESIGN})	0.025200	6.800900	0.362578

Now, lets explain another FP-Growth Example with NYC Travel Dataset

```

fp_growth_nyc.py x dataset_TSMC2014_NYC.csv
1 import pandas as pd
2 from mlxtend.frequent_patterns import fpgrowth
3 from mlxtend.preprocessing import TransactionEncoder
4 from mlxtend.frequent_patterns import association_rules
5
6 pd.set_option('display.max_columns', None) # maximum column limit
7 pd.set_option("display.max_colwidth", None)
8 pd.set_option('display.width', 1000)
9
10 df = pd.read_csv('dataset_TSMC2014_NYC.csv')
11 print("Dataset successfully loaded.")
12 print("Dataset shape:", df.shape)
13
14 print("\n---First 5 Rows---")
15 print(df.head())
16
17 print("\n---Dataset Information---")
18 print(df.info())
19
20 print("\n--- Descriptive Statistics ---")
21 print(df.describe())
22
23 print("\n--- Missing Values Check ---")
24 print(df.isnull().sum())
25
26 # drop if dataset has duplicate values
27 df = df.drop_duplicates()
28
29 # drop if venueCategory has 'NaN'
30 df["venueCategory"] = df["venueCategory"].replace(to_replace=["NaN", "nan", "NAN", ""], pd.NA)
31 df = df.dropna(subset=["venueCategory"])
32

```

```

32
33 # Not considering non-touristic locations
34 non_touristic_categories = [
35     "Medical Center",
36     "Subway",
37     "Office",
38     "Student Center",
39     "Bus Station",
40     "Home (private)",
41     "NaN"
42 ]
43
44 df = df[~df["venueCategory"].isin(non_touristic_categories)]
45
46 df = df[~df["venueCategory"].str.contains(pat="Restaurant",na=False)]
47 # drop where venueCategory include Restaurant
48
49 print("\n---Dataset after removing non-touristic categories:---")
50 print(df.head(5))
51 print("\n")
52
53 df = df[["userId", "venueCategory"]] # only necessary columns
54
55 # group userId by venueCategory
56 transactions = df.groupby("userId")["venueCategory"].apply(lambda x: list(dict.fromkeys(x))).tolist()
57 print(transactions[:5])
58 print("\n")
59
60 transaction_encoder = TransactionEncoder()
61 transaction_encoder.fit(transactions)
62 encoded_transactions = transaction_encoder.transform(transactions)
63 # show list in True / False format

```

```

df_pd = pd.DataFrame(encoded_transactions, columns=transaction_encoder.columns_)
print("Dataset after TransactionEncoder:")
print(df_pd.head())
print("\n")

frequent_itemSets = fpgrowth(df_pd, min_support=0.05, use_colnames=True, max_len=3)
# min_support means that items must appear in at least 5% of transactions
# maximum 3 combinations allowed

frequent_itemSets = frequent_itemSets.sort_values(by=['support'], ascending=False)
# sort item sets by their support descending

print("---Frequent Item Sets---")
print(frequent_itemSets.head())
print("\n")

rules = association_rules(frequent_itemSets, metric='lift', min_threshold=1.2)
# lift must be at least 1 or higher

```



```

rules["antecedents"] = rules["antecedents"].apply(lambda x: ", ".join(list(x)))
rules["consequents"] = rules["consequents"].apply(lambda x: ", ".join(list(x)))
# remove frozenset

print("Item Combinations using FP-Growth")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head(15))
print("\n")

```

Travel Dataset has columns such as userId, venueId, and venueCategory.

Dataset also has longitude and latitude but they are not necessary for this project because we don't need to find locations.

---First 5 Rows---

	userId	venueId	venueCategoryId	venueCategory	latitude	longitude	timezoneOffset	utcTimestamp
0	470	49bbd6c0f964a520f4531fe3	4bf58dd8d48988d127951735	Arts & Crafts Store	40.719810	-74.002581	-240 Tue Apr 03 18:00:09 +0000 2012	
1	979	4a43c0aef964a520c6a61fe3	4bf58dd8d48988d1df941735	Bridge	40.606800	-74.044170	-240 Tue Apr 03 18:00:25 +0000 2012	
2	69	4c5cc7b485a1e21e00d35711	4bf58dd8d48988d103941735	Home (private)	40.716162	-73.883070	-240 Tue Apr 03 18:02:24 +0000 2012	
3	395	4bc7086715a7ef3bef9878da	4bf58dd8d48988d104941735	Medical Center	40.745164	-73.982519	-240 Tue Apr 03 18:02:41 +0000 2012	
4	87	4cf2c5321d18a143951b5cec	4bf58dd8d48988d1cb941735	Food Truck	40.740104	-73.989658	-240 Tue Apr 03 18:03:00 +0000 2012	

This example is mostly same with previous FP-Growth example but this time we did much more Data Preparation.

Here is the final output:

```

Item Combinations using FP-Growth

```

	antecedents	consequents	support	confidence	lift
0	Coffee Shop, Food & Drink Shop	Bakery	0.331487	0.529499	1.220100
1	Bakery	Coffee Shop, Food & Drink Shop	0.331487	0.763830	1.220100
2	Food & Drink Shop, Clothing Store	Department Store	0.314866	0.620000	1.225292
3	Department Store	Food & Drink Shop, Clothing Store	0.314866	0.622263	1.225292
4	Food & Drink Shop, Pizza Place	Deli / Bodega	0.311173	0.568297	1.209166
5	Deli / Bodega	Food & Drink Shop, Pizza Place	0.311173	0.662083	1.209166
6	Airport	Hotel	0.294552	0.552860	1.202303
7	Hotel	Airport	0.294552	0.640562	1.202303
8	Coffee Shop, Park	Bakery	0.292705	0.532773	1.227645
9	Bakery	Coffee Shop, Park	0.292705	0.674468	1.227645
10	Bar, Clothing Store	Bakery	0.291782	0.521452	1.201559
11	Bar, Bakery	Clothing Store	0.291782	0.736597	1.212362
12	Clothing Store	Bar, Bakery	0.291782	0.480243	1.212362
13	Bakery	Bar, Clothing Store	0.291782	0.672340	1.201559
14	Food & Drink Shop, Clothing Store	Drugstore / Pharmacy	0.290859	0.572727	1.253058

After FP-Growth examples, ECLAT Algorithm is explained at the next page.

ECLAT Algorithm

ECLAT stands for Equivalence Class Clustering and bottom-up Lattice Traversal. It is a data mining algorithm used to find frequent itemsets in a dataset. These frequent itemsets are then used to create association rules which helps to identify patterns in data.

How it works:

Imagine a transaction boolean dataset

Transaction ID	Bread	Butter	Milk	Coke	Jam
T1	1	1	0	0	1
T2	0	1	0	1	0
T3	0	1	1	0	0
T4	1	1	0	1	0
T5	1	0	1	0	0
T6	0	1	1	0	0
T7	1	0	1	0	0
T8	1	1	1	0	1
T9	1	1	1	0	0

Here are 9 transaction with each of them with item or multiple items.

Now, lets create a Tidset to show each item with transactions they are in.

Lets give $k=1$ and minimum support = 2

Item	Tidset
Bread	{T1, T4, T5, T7, T8, T9}
Butter	{T1, T2, T3, T4, T6, T8, T9}
Milk	{T3, T5, T6, T7, T8, T9}
Coke	{T2, T4}
Jam	{T1, T8}

Values that are smaller than 2 Tidset will drop but since each item has at least 2 transactions, none of them are deleted.

$K=2$, min_support = 2

Item	Tidset
{Bread, Butter}	{T1, T4, T8, T9}
{Bread, Milk}	{T5, T7, T8, T9}
{Bread, Coke}	{T4}
{Bread, Jam}	{T1, T8}
{Butter, Milk}	{T3, T6, T8, T9}
{Butter, Coke}	{T2, T4}
{Butter, Jam}	{T1, T8}
{Milk, Jam}	{T8}

The algorithm continues recursively by combining pairs of itemsets (k-itemsets) checking the support by intersecting the tidsets. The recursion continues until no further frequent itemsets can be generated. Now $k = 3$.

Item	Tidset
{Bread, Butter, Milk}	{T8, T9}
{Bread, Butter, Jam}	{T1, T8}

The algorithm stops once no more itemset combinations meet the minimum support threshold. $k = 4$

Item	Tidset
{Bread, Butter, Milk, Jam}	{T8}

We stop at $k = 4$ because there are no more item-tidset pairs to combine. Since minimum support = 2, we conclude the following rules from the given dataset:

Items Bought	Recommended Products
Bread	Butter
Bread	Milk
Bread	Jam
Butter	Milk
Butter	Coke
Butter	Jam
Bread and Butter	Milk
Bread and Butter	Jam

Here is an real Python example:

```
eclat_example.py x
1  from collections import defaultdict
2  from itertools import combinations
3  # libraries necessary for ECLAT
4
5  transactions = {
6      "T1": ["Bread", "Butter", "Jam"],
7      "T2": ["Butter", "Coke"],
8      "T3": ["Butter", "Milk"],
9      "T4": ["Bread", "Butter", "Coke"],
10     "T5": ["Bread", "Milk"],
11     "T6": ["Butter", "Milk"],
12     "T7": ["Bread", "Milk"],
13     "T8": ["Bread", "Butter", "Milk", "Jam"],
14     "T9": ["Bread", "Butter", "Milk"]
15 }
16
17 min_support = 2 # items must be at least in 2 transaction
18
19 def generate_tidsets(transactions: {items}): 2+ usages
20     item_tidset = defaultdict(set)
21     for tid, items in transactions.items(): # go through every transaction
22         for item in items: # go through every item inside transactions
23             item_tidset[item].add(tid) # ex: item_tidset['Bread'].add('T1')
24     return item_tidset
25
26 item_tidset = generate_tidsets(transactions)
27
28 for item, tidset in item_tidset.items():
29     print(item, ': ', sorted(tidset)) # print the list with each item showing which transactions they belong
30
31 print("\n")
32
33 # show the items that have least transactions first
34 items = sorted(item_tidset.items(), key=lambda x: len(x[1]))
35
36 def eclat(prefix, items: {pop}, min_support, frequent_items): 2+ usages
37     while items: # continue until all items are searched
38         item, tidset = items.pop() # take one item and its transaction ID's
39         support = len(tidset) # amount of transactions that item has
40
41         if support >= min_support: # must be bigger than 2
42             new_itemset = prefix + [item] # create new itemset by adding current item to prefix
43             frequent_items[frozenset(new_itemset)] = support
44             suffix = []
45
```

```

45
46     for other_item, other_tidset in items: # compare current itemset to other items
47         intersection = tidset & other_tidset # Find transactions that contain BOTH items.
48
49         if len(intersection) >= min_support: # ex: {Bread, Butter} : {T1,T3,T4} --> 3>2
50             suffix.append((other_item, intersection)) # if combined items transaction number > 2, append it to suffix list
51         suffix = sorted(suffix, key=lambda x : (_getitem_): len(x[1])) # sort suffix by support count
52
53     eclat(new_itemset, suffix, min_support, frequent_items)
54
55 item_tidset = generate_tidsets(transactions)
56 items = sorted(item_tidset.items(), key=lambda x : (_getitem_): len(x[1]))
57 frequent_items = {}
58 eclat( prefix: [], items, min_support, frequent_items)
59
60 print("Frequent Itemsets (as list) -> support count")
61 for itemset, support in sorted(frequent_items.items(), key=lambda x : (_getitem_): (-len(x[0]), -x[1], sorted(list(x[0])))):
62     print(list(itemset), "=>", support)

```

Collections library is added to implement ECLAT Algorithm.

Transaction list is defined and minimum support is 2.

Generate_tidsets() function converts the transaction dataset into this vertical representation.

The function visits every transaction and every product inside that transaction. It then stores the transaction ID inside the corresponding product's set.

Eclat function is implemented to update Tidset until every item is searched.

Frequent_items is a dictionary where we keep discovered frequent item sets.

Support is the amount of transaction that item has. It must be bigger than min_support which is 2.

In this function, we also find intersections, transactions that contain both items.

We created list named suffix to add these intersections to it.

Eclat function is called recursively each time until there is no item left to search.

At the next page, I will explain ECLAT with an real dataset.

ECLAT With Movies Dataset

```
1
2     import pandas as pd
3     from pyECLAT import ECLAT
4
5     movies = pd.read_csv('movies.csv')
6     ratings = pd.read_csv('ratings.csv')
7
8     pd.set_option('display.max_columns', 6)
9     pd.set_option('display.max_rows', 50)
10    pd.set_option('display.width', 200)
11
12    print("Movies Dataset")
13    print(movies.head())
14    print("\n")
15
16    print("Movies Information")
17    print(movies.info())
18    print("\n")
19
20    print("Ratings Dataset")
21    print(ratings.head())
22    print("\n")
23
24    print("Ratings Information")
25    print(ratings.info())
26    print("\n")
27
28    print("Movies Missing Information")
29    print(movies.isnull().sum())
30    print("\n")
31
32    print("Ratings Missing Information")
33    print(ratings.isnull().sum())
34    print("\n")
35
36    print("User Amount")
37    print(ratings['userId'].nunique())
38    print("\n")
```

```

40 print("Movies Amount")
41 print(movies['movieId'].nunique())
42 print("\n")
43
44 # ----- #
45
46 ratings = ratings[ratings['rating'] >= 4]
47
48 for col in ratings.columns:
49     accepted_ratings = (ratings['rating'] >= 4).sum()
50
51 print("Ratings that are bigger than 3")
52 print(accepted_ratings)
53 print("\n")
54
55 df_merged = ratings.merge(movies, on='movieId')
56 print("Merged Dataset First 5 Rows")
57 print(df_merged.head())
58 print("\n")
59
60 # keep only movies liked by at least 50 users
61 movie_like_counts = df_merged.groupby('title')['userId'].nunique()
62 popular_movies = movie_like_counts[movie_like_counts >= 50].index
63 df_merged = df_merged[df_merged['title'].isin(popular_movies)]
64
65 # sort by userId and rating
66 df_merged = df_merged.sort_values(by=['userId', 'rating'], ascending=[True, False])
67
68 # keep maximum 20 liked movies for each user
69 df_merged = df_merged.groupby('userId').head(20)
70
71 df_merged = df_merged.drop(columns=['rating', 'timestamp', 'movieId', 'genres'])
72 transactions = df_merged.groupby('userId')['title'].apply(list).tolist()
73
74 print("Merged Dataset After Grouping UserId by Title")
75 print(transactions[:5])
76 print("\n")

```

```

78 df_transactions = pd.DataFrame(transactions)
79 print("Transactions Dataset For ECLAT Algorithm")
80 print(df_transactions[:5])
81
82 eclat_instance = ECLAT(data=df_transactions, verbose=True)
83
84 rule_indices, rule_supports = eclat_instance.fit(
85     min_support=0.1,
86     min_combination=2,
87     max_combination=2,
88     separator=' & ',
89     verbose=True
90 )
91
92 print("Frequent Movie Combinations")
93 sorted_rules = sorted(rule_supports.items(), key=lambda x: {__getitem__}: x[1], reverse=True)
94
95 for rule, support in sorted_rules[:10]:
96     print(rule, "Support:", support)
97
98 print("\n")

```



```

100 def recommend_movie(): 1 usage
101     recommendations = []
102     movie_name = input("Enter movie name:")
103
104     for rule, support in rule_supports.items():
105         movies_in_rule = rule.split(' & ')
106
107         if movie_name in movies_in_rule:
108             for movie in movies_in_rule:
109                 if movie != movie_name:
110                     recommendations.append((movie,support))
111
112     if len(recommendations) == 0:
113         print("No recommendation found for this movie.")
114         return
115
116     recommendations = sorted(recommendations, key=lambda x : {__getitem__} : x[1], reverse=True)
117
118     print("Movie recommendations:")
119     print("----" * 10)
120
121     for movie, support in recommendations[:10]:
122         print(movie, "Support:", support)
123
124     recommend_movie()
125     print("\n")

```

PyECLAT library is added to implement ECLAT Algorithm.

Firstly, Dataset information is printed out.

Data Preparation process steps:

Ratings in rating dataset must be bigger than 3. Then ratings that are bigger than 3 is printed out.

Movies and ratings dataset is merged by using movieId column that both had.

Only movies that liked by at least 50 people is kepted.

Selected highest rated movies from each user.

The amount of liked movie limit is 20 for each user so ECLAT Algorithms works faster.

Unnecessary columns are dropped.

Created User transactions which grouped by userId.

```

Merged Dataset After Grouping UserId by Title
[['Seven (a.k.a. Se7en) (1995)', 'Usual Suspects, The (1995)', 'Star Wars: Episode IV - A New Hope (1977)',

```

After that, transactions converted into a Dataframe.

```
eclat_instance = ECLAT(data=df_transactions, verbose=True)

rule_indices, rule_supports = eclat_instance.fit(
    min_support=0.1,
    min_combination=2,
    max_combination=2,
    separator=' & ',
    verbose=True
)
```

Here is how pyECLAT library is implemented.

Frequent Movie Combinations are printed out, here is some part of the output:

```
Frequent Movie Combinations
Forrest Gump (1994) & Shawshank Redemption, The (1994) Support: 0.2184873949579832
Shawshank Redemption, The (1994) & Pulp Fiction (1994) Support: 0.21008403361344538
Shawshank Redemption, The (1994) & Schindler's List (1993) Support: 0.16638655462184873
Forrest Gump (1994) & Pulp Fiction (1994) Support: 0.16302521008403362
Star Wars: Episode IV - A New Hope (1977) & Star Wars: Episode V - The Empire Strikes Back (1980) Support: 0.15630252100840336
Silence of the Lambs, The (1991) & Pulp Fiction (1994) Support: 0.15630252100840336
Shawshank Redemption, The (1994) & Usual Suspects, The (1995) Support: 0.1546218487394958
Silence of the Lambs, The (1991) & Shawshank Redemption, The (1994) Support: 0.15294117647058825
```

Lastly, recommendation function is added so user can enter a movie to see which other movies are associated with it.

Here is an example output:

```
Enter movie name:Forrest Gump (1994)
Movie recommendations:
-----
Shawshank Redemption, The (1994) Support: 0.2184873949579832
Pulp Fiction (1994) Support: 0.16302521008403362
Braveheart (1995) Support: 0.12773109243697478
Silence of the Lambs, The (1991) Support: 0.12773109243697478
Schindler's List (1993) Support: 0.12100840336134454
```

Conclusion

In this report, FP-Growth and ECLAT algorithms were examined as two important methods used in association mining. Both algorithms aim to discover frequent itemsets and identify items that commonly appear together. However, they use different data structures and processing methods to achieve this goal.

FP-Growth and ECLAT both produced useful frequent itemsets, but the selection of the algorithm depends on the structure and size of the dataset. FP-Growth is generally suitable when transaction data can be efficiently compressed into an FP-tree, while ECLAT can be useful when vertical transaction sets and intersection operations can be processed effectively.

Many examples are implemented and variety of datasets are used such as Travel, Online Retail and Movies dataset.

Overall, the four examples provided practical experience in applying FP-Growth and ECLAT to different types of data. The results showed how frequent patterns can be discovered and interpreted to support recommendation systems and behavioral analysis.